DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COURSE: CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS
CO3 – CASE STUDY REPORT

DIVIDE-AND-CONQUER ENGINE OF "KARTIQ" A RETAIL DATA ANALYTICS PLATFORM

Submitted by
Name: ASWINI S
Register Number: 192511077

Submitted to
Department of Computer Science and Engineering

DECLARATION

I hereby declare that this report entitled “Divide-and-Conquer Engine of KartIQ: A Retail Data Analytics Platform” is prepared by me as part of the CSA06 – Design and Analysis of Algorithms CO3 case study assessment. The explanations, algorithm designs, pseudocode, program implementations and sample outputs are presented for academic purposes.

Student Name: ASWINI S

Register Number:192511077

ACKNOWLEDGEMENT

I express my sincere gratitude to the Department of Computer Science and Engineering and the course faculty for providing the opportunity to complete this case study on divide-and-conquer algorithms. This work helped me understand how algorithm selection affects the performance, memory usage and reliability of large-scale data analytics systems.

ABSTRACT

KartIQ is a national e-commerce analytics platform that processes approximately 10,000,000 orders per day and maintains a leaderboard for 100,000 sellers. The platform experiences delays in nightly order consolidation, freezes during leaderboard refreshes, slow order lookups and infrequent recommendation-score updates. This report studies four computational modules and redesigns them using divide-and-conquer techniques.

Stable Merge Sort is selected for ordering orders by timestamp, ensuring that orders with equal timestamps retain their original arrival order. A randomized three-way Quick Sort strategy is proposed for seller scores to reduce the risk of highly unbalanced partitions. Binary Search is modified to locate the first order matching a timestamp in the sorted order list. Strassen’s matrix multiplication is examined for recommendation-score computation, with a practical hybrid implementation that switches to conventional multiplication for small submatrices.

For every module, the report presents the problem, design decisions, pseudocode, Python implementation, sample execution, trace, recurrence relation, time and space complexity, and practical limitations. The final design combines the four algorithms into a coherent analytics pipeline and explains the trade-offs involved.

TABLE OF CONTENTS

- 1. Introduction and Case Study Overview
- 2. Understanding the Case and Algorithm Requirements
- 3. Module 1 – Stable Merge Sort for Nightly Order Consolidation
- 4. Module 2 – Improved Quick Sort for Live Seller Leaderboard
- 5. Module 3 – First-Occurrence Binary Search for Order Lookup
- 6. Module 4 – Strassen’s Matrix Multiplication for Recommendations
- 7. Integrated Implementation and Sample Outputs
- 8. Comparative Analysis and Final Design

- 9. Conclusion
- 10. References and Appendix

Note: Page numbers may be updated in Microsoft Word after opening the document.

1. INTRODUCTION AND CASE STUDY OVERVIEW

1.1 Introduction

Divide and conquer is an algorithm design technique in which a large problem is divided into smaller subproblems of the same type. Each subproblem is solved recursively, and the partial results are combined to obtain the final answer. This approach is useful when a problem can be decomposed into independent or smaller tasks.

The general stages are: (i) Divide – split the input into smaller parts; (ii) Conquer – solve each part, often recursively; and (iii) Combine – merge the solutions. Merge Sort, Quick Sort, Binary Search and Strassen's matrix multiplication are important examples.

1.2 KartIQ Platform Background

The case describes KartIQ, a national e-commerce marketplace analytics platform. It records about 10 million orders each day and ranks 100,000 sellers. The platform faces four performance problems: the nightly order report misses its deadline, the live seller leaderboard freezes during sale events, customer-support order lookups take seconds, and recommendation scores are refreshed only once a day.

1.3 Objectives

- Identify the computational requirements and algorithmic risks in all four modules.
- Select suitable divide-and-conquer algorithms and justify each selection.
- Trace the algorithms using the sample data supplied in the case study.
- Formulate and solve recurrence relations and estimate operation counts.
- Implement the algorithms in Python and show representative outputs.
- Combine the algorithms into a practical design for the KartIQ platform.

1.4 Scope and Assumptions

The sample data and production sizes used in this report are taken from the supplied case study. The implementation is a reference Python version intended to demonstrate correctness and algorithm behaviour. Production performance depends on hardware, programming language, memory, data distribution, concurrency and storage architecture. The output values shown below are deterministic sample outputs from the stated input; large-scale operation counts are theoretical estimates, not measured benchmark timings.

2. UNDERSTANDING THE CASE AND ALGORITHM REQUIREMENTS

Each module has a different requirement. The correct algorithm must satisfy functional needs as well as performance and memory constraints.

Module	Computational requirement	Key issue	Risk of poor choice
1. Order consolidation	Sort 10 million orders by timestamp	Stability for equal timestamps; auxiliary memory	Audit order may change; report may miss deadline
2. Seller leaderboard	Re-sort 100,000 seller scores repeatedly	Nearly sorted input; pivot quality; worst-case time	First-pivot Quick Sort can degrade to quadratic time and freeze refreshes
3. Order lookup	Find timestamp in sorted orders	Sorted-data precondition; return first duplicate	Linear scan may take millions of comparisons
4. Recommendation scores	Multiply two 1,024 × 1,024 matrices	Large matrix size; recursion overhead; memory	Conventional multiplication may be costly; pure Strassen may incur overhead

2.1 Why Divide and Conquer?

The four tasks are suitable for divide-and-conquer because sorting can be performed on smaller portions and merged, searching can repeatedly discard half the remaining range, and matrix multiplication can split matrices into blocks. These approaches reduce the amount of work compared with simple repeated scans or unsuitable implementations.

2.2 Design Principles

- Correctness first: preserve order where the business requirement explicitly demands it.
- Predictable performance: avoid algorithm choices that have a severe worst case on the expected data.
- Memory awareness: account for temporary arrays and matrix blocks, not only running time.
- Production practicality: use hybrid techniques where recursion overhead can outweigh theoretical gains.
- Reproducibility: show sample inputs and outputs and avoid presenting theoretical estimates as measured timings.

3. MODULE 1 – STABLE MERGE SORT FOR NIGHTLY ORDER CONSOLIDATION

3.1 Problem Description

KartIQ must sort all daily orders by timestamp to generate the audit report. The supplied sample is (105, O1), (102, O2), (105, O3), (101, O4), (102, O5), (103, O6). Orders that have the same timestamp must keep their original arrival order because auditors reconcile transactions gateway by gateway.

3.2 Algorithm Selection and Justification

Stable Merge Sort is selected. It divides the list into halves, recursively sorts each half, and merges the sorted halves. During merging, when two timestamps are equal, the item from the left half is selected first. Since the left half contains earlier elements from the original input, this tie rule preserves arrival order.

- Time complexity: O(n log n) in best, average and worst cases.
- Auxiliary space: O(n) for an array-based implementation.
- Stability: stable when equal keys are taken from the left run first.
- Suitability: predictable performance for 10,000,000 orders.

3.3 Pseudocode

```
MERGE-SORT(A):
    if length(A) <= 1:
        return A
    middle = floor(length(A) / 2)
    left = MERGE-SORT(A[0 : middle])
    right = MERGE-SORT(A[middle : length(A)])
    return MERGE(left, right)

MERGE(left, right):
    result = empty list
    i = 0; j = 0
    while i < length(left) and j < length(right):
        if left[i].timestamp <= right[j].timestamp:
            append left[i] to result
            i = i + 1
        else:
            append right[j] to result
            j = j + 1
    append remaining elements of left to result
    append remaining elements of right to result
    return result
```

3.4 Step-by-Step Trace

Initial sequence:

(105,O1), (102,O2), (105,O3), (101,O4), (102,O5), (103,O6)

Division:

- Left half: [(105,O1), (102,O2), (105,O3)]
- Right half: [(101,O4), (102,O5), (103,O6)]
- Continue splitting until each sublist contains one order.

Merge operations:

Merge step	Left run	Right run	Merged result
1	[(105,O1)]	[(102,O2)]	[(102,O2),(105,O1)]

2	[(102,O2),(105,O1)]	[(105,O3)]	[(102,O2),(105,O1),(105,O3)]
3	[(101,O4)]	[(102,O5)]	[(101,O4),(102,O5)]
4	[(101,O4),(102,O5)]	[(103,O6)]	[(101,O4),(102,O5),(103,O6)]
5	[(102,O2),(105,O1),(105,O3)]	[(101,O4),(102,O5),(103,O6)]	[(101,O4),(102,O2),(102,O5),(103,O6),(105,O1),(105,O3)]

Final sorted order: (101,O4), (102,O2), (102,O5), (103,O6), (105,O1), (105,O3). O2 remains before O5 for timestamp 102, and O1 remains before O3 for timestamp 105. Therefore, stability is maintained.

3.5 Python Implementation

```
def merge_sort_orders(orders):
    """Stable merge sort by timestamp; original order breaks timestamp ties."""
    if len(orders) <= 1:
        return orders[:]

    mid = len(orders) // 2
    left = merge_sort_orders(orders[:mid])
    right = merge_sort_orders(orders[mid:])

    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][0] <= right[j][0]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged


orders = [(105, "O1"), (102, "O2"), (105, "O3"),
          (101, "O4"), (102, "O5"), (103, "O6")]
print(merge_sort_orders(orders))
```

3.6 Sample Output

```
[(101, 'O4'), (102, 'O2'), (102, 'O5'),
 (103, 'O6'), (105, 'O1'), (105, 'O3')]
```

3.7 Recurrence and Complexity Analysis

For n greater than 1, Merge Sort performs two recursive calls on approximately n/2 elements and spends Θ(n) time merging:

$T(n) = 2T(n/2) + cn$, with $T(1) = \Theta(1)$.

By the Master Theorem, a = 2, b = 2 and $f(n) = \Theta(n)$. Since $n^{(\log_2 2)} = n$, this is Case 2, giving $T(n) = \Theta(n \log_2 n)$.

For 10,000,000 orders, $\log_2(10{,}000{,}000)$ is approximately 23.2535. The $n \log_2 n$ scale is approximately 232.5 million. A conservative upper bound on merge comparisons for a balanced merge sort is $n \times \text{ceil}(\log_2 n) - 2^{\text{ceil}(\log_2 n)} + 1$, which is 223,222,785 for n = 10,000,000. Actual comparisons depend on the data and merge boundaries.

The array-based implementation needs O(n) additional storage. Ten million references and associated order objects can require substantial memory. If the complete dataset does not fit in RAM, an external merge sort

can sort manageable chunks on disk, write sorted runs, and merge those runs using buffered sequential reads and writes.

3.8 Module 1 Conclusion

Stable Merge Sort meets the audit requirement because it provides predictable O(n log n) sorting and preserves the original order of equal timestamps. Its principal trade-off is auxiliary memory. External merge sorting is the practical alternative when the full order set cannot fit in memory.

4. MODULE 2 – IMPROVED QUICK SORT FOR LIVE SELLER LEADERBOARD

4.1 Problem Description

The seller leaderboard contains 100,000 seller scores and is refreshed every few seconds. Between refreshes, only a few scores change, so the list is nearly sorted. The current implementation chooses the first element as the pivot. This strategy can repeatedly create highly unbalanced partitions on already sorted or nearly sorted data.

4.2 Partition Trace on the Supplied Sample

Sample scores: [42, 17, 63, 8, 55, 29]. Using the first element, 42, as a Lomuto-style pivot, move values less than or equal to 42 to the left and larger values to the right.

Action	Array state
Initial	[42, 17, 63, 8, 55, 29]
Compare 17 with 42; move left	[42, 17, 63, 8, 55, 29]
Compare 63; leave on right	[42, 17, 63, 8, 55, 29]
Compare 8; move left	[42, 17, 8, 63, 55, 29]
Compare 55; leave on right	[42, 17, 8, 63, 55, 29]
Compare 29; move left	[42, 17, 8, 29, 55, 63]
Place pivot in final position	[29, 17, 8, 42, 55, 63]

The pivot 42 is now in its final sorted position. The left partition is [29,17,8] and the right partition is [55,63].

4.3 Why First-Element Pivot Is a Problem

If the input is already sorted and the first item is always selected as pivot, one partition has zero elements and the other has n−1 elements. The recursion depth becomes n and the total comparisons become approximately n(n−1)/2. For 100,000 sellers this is 4,999,950,000 comparisons in the fully sorted worst-case pattern. Nearly sorted data can produce similarly poor partitions.

4.4 Improved Design

Use a randomized pivot together with three-way partitioning. Randomization reduces dependence on the input order, while three-way partitioning groups values smaller than, equal to and greater than the pivot. This is useful when several sellers have equal scores. In an environment where reproducible runs are required, a fixed random seed can be used for testing.

4.5 Pseudocode

```
QUICK-SORT-3-WAY(A, low, high):
    if low >= high:
        return
    choose a random pivot from A[low : high + 1]
    swap pivot into A[low]
    pivot = A[low]
    lt = low; i = low + 1; gt = high

    while i <= gt:
        if A[i] < pivot:
            swap A[lt] and A[i]
            lt = lt + 1
            i = i + 1
        else if A[i] > pivot:
            swap A[i] and A[gt]
            gt = gt - 1
        else:
```

```
            i = i + 1

    QUICK-SORT-3-WAY(A, low, lt - 1)
    QUICK-SORT-3-WAY(A, gt + 1, high)
```

4.6 Python Implementation

```
import random

def quick_sort_3way(values, seed=7):
    """Return a sorted copy using randomized three-way Quick Sort."""
    a = values[:]
    rng = random.Random(seed)

    def sort(low, high):
        if low >= high:
            return

        pivot_index = rng.randint(low, high)
        a[low], a[pivot_index] = a[pivot_index], a[low]
        pivot = a[low]

        lt = low
        i = low + 1
        gt = high

        while i <= gt:
            if a[i] < pivot:
                a[lt], a[i] = a[i], a[lt]
                lt += 1
                i += 1
            elif a[i] > pivot:
                a[i], a[gt] = a[gt], a[i]
                gt -= 1
            else:
                i += 1

        sort(low, lt - 1)
        sort(gt + 1, high)

    sort(0, len(a) - 1)
    return a


scores = [42, 17, 63, 8, 55, 29]
print(quick_sort_3way(scores))
```

4.7 Sample Output

```
[8, 17, 29, 42, 55, 63]
```

4.8 Recurrences and Complexity

Best case: $T(n) = 2T(n/2) + cn$, which solves to $\Theta(n \log n)$.

Worst case: $T(n) = T(n-1) + cn$, which solves to $\Theta(n^2)$. Randomized pivot selection makes consistently unbalanced partitions less likely, but it does not mathematically eliminate the worst case.

Expected time for randomized Quick Sort is $\Theta(n \log n)$. For 100,000 sellers, $n \log_2 n$ is approximately 1,660,964 comparison-scale operations, while the exact count depends on the pivot sequence and input. This is an order-of-growth estimate, not a measured result.

Expected auxiliary stack space is $O(\log n)$ for reasonably balanced recursion; worst-case recursion depth is $O(n)$. Three-way partitioning can be particularly effective when many scores are equal.

4.9 Practical Considerations

- An introspective sort can provide a stronger worst-case guarantee by switching to heapsort when recursion becomes too deep.
- If only a few scores change, an incremental data structure such as a balanced search tree or heap may avoid sorting the entire leaderboard on every refresh.
- Quick Sort is generally in-place apart from recursion stack usage, but the implementation above returns a copy to avoid modifying the caller's input.

4.10 Module 2 Conclusion

Randomized three-way Quick Sort is a better replacement than first-element-pivot Quick Sort for the stated nearly sorted leaderboard. It offers expected O(n log n) behaviour and handles duplicate scores efficiently. For a strict worst-case service-level requirement, introsort or an incremental ranking structure should also be considered.

5. MODULE 3 – FIRST-OCCURRENCE BINARY SEARCH FOR ORDER LOOKUP

5.1 Problem Description

Customer-support staff search for an order using its timestamp in the consolidated list produced by Module 1. Since multiple orders may have the same timestamp, the search must return the first order with that timestamp.

5.2 Why Sorted Data Is Essential

Binary Search works only when the search range is sorted according to the same key being compared. Module 1 produces orders in non-decreasing timestamp order. This allows each comparison to eliminate approximately half of the remaining range.

5.3 Modified Binary Search

To return the first occurrence, when the middle timestamp equals the target, record the middle position and continue searching the left half. If the middle timestamp is smaller than the target, search right; otherwise search left. The stored candidate is the earliest matching index found.

5.4 Pseudocode

```
FIRST-OCCURRENCE-BINARY-SEARCH(A, target):
    low = 0
    high = length(A) - 1
    answer = -1

    while low <= high:
        mid = floor((low + high) / 2)
        if A[mid].timestamp == target:
            answer = mid
            high = mid - 1
        else if A[mid].timestamp < target:
            low = mid + 1
        else:
            high = mid - 1

    return answer
```

5.5 Python Implementation

```
def first_timestamp_index(orders, target):
    """Return the first index for target timestamp, or -1 if absent."""
    low = 0
    high = len(orders) - 1
    answer = -1

    while low <= high:
        mid = (low + high) // 2
        timestamp = orders[mid][0]

        if timestamp == target:
            answer = mid
            high = mid - 1
        elif timestamp < target:
            low = mid + 1
        else:
            high = mid - 1

    return answer


sorted_orders = [
    (101, "O4"), (102, "O2"), (102, "O5"),
    (103, "O6"), (105, "O1"), (105, "O3")
]
index = first_timestamp_index(sorted_orders, 102)
print(index)
```

```
print(sorted_orders[index] if index != -1 else "Not found")
```

5.6 Sample Search Trace

Search target: 102. Sorted timestamps: [101, 102, 102, 103, 105, 105].

Iteration	low	high	mid	Timestamp at mid	Decision
1	0	5	2	102	Match; save index 2 and continue left
2	0	1	0	101	Smaller than target; search right
3	1	1	1	102	Match; save index 1 and continue left
Stop	1	0	–	–	Range is empty; return index 1

The first matching order is at index 1: (102, O2). The later duplicate (102, O5) is not returned.

5.7 Sample Output

```
1
(102, 'O2')
```

5.8 Recurrence and Complexity

At each step, Binary Search reduces the problem size by approximately half and performs constant work:

$T(n) = T(n/2) + c$, with $T(1) = \Theta(1)$.

By the Master Theorem, $T(n) = \Theta(\log_2 n)$. The modified first-occurrence search may continue after finding a match, but it still takes O(log n) time.

For 10,000,000 records, the maximum number of iterations is $\text{ceil}(\log_2(n+1)) = 24$. Thus, at most 24 midpoint comparisons are needed under the standard comparison-count bound. The algorithm uses O(1) extra space.

5.9 Module 3 Conclusion

First-occurrence Binary Search provides fast timestamp lookup and correctly handles duplicate timestamps. Its correctness depends on the sorted output of Module 1. It reduces the lookup work from O(n) for a linear scan to O(log n).

6. MODULE 4 – STRASSEN'S MATRIX MULTIPLICATION FOR RECOMMENDATIONS

6.1 Problem Description

KartIQ computes recommendation scores using C = A × B, where A is the segment–feature preference matrix and B is the feature–product matrix. In production, both matrices are 1,024 × 1,024. The supplied sample matrices are A = [[1,7],[3,5]] and B = [[1,3],[7,5]].

6.2 Conventional Matrix Multiplication

Conventional multiplication computes each output entry as the dot product of a row of A and a column of B. For two n × n matrices it performs n^3 scalar multiplications and $\Theta(n^3)$ arithmetic work.

6.3 Strassen's Seven Products

Split each matrix into four equal-sized blocks:

A = [[A11, A12], [A21, A22]] and B = [[B11, B12], [B21, B22]].

Strassen's method uses seven recursive products instead of eight:

```
M1 = (A11 + A22) (B11 + B22)
M2 = (A21 + A22) B11
M3 = A11 (B12 - B22)
M4 = A22 (B21 - B11)
M5 = (A11 + A12) B22
M6 = (A21 - A11) (B11 + B12)
M7 = (A12 - A22) (B21 + B22)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6
```

6.4 Verification Using the 2 × 2 Sample

A = [[1,7],[3,5]] and B = [[1,3],[7,5]]. Treat each element as a 1 × 1 block:

M1 = (1+5)(1+5) = 36

M2 = (3+5)(1) = 8

M3 = 1(3−5) = −2

M4 = 5(7−1) = 30

M5 = (1+7)(5) = 40

M6 = (3−1)(1+3) = 8

M7 = (7−5)(7+5) = 24

C11 = 36+30−40+24 = 50; C12 = −2+40 = 38; C21 = 8+30 = 38; C22 = 36−8−2+8 = 34.

Therefore, C = [[50,38],[38,34]], which matches conventional multiplication.

6.5 Pseudocode

```
STRASSEN(A, B):
    n = number of rows in A
    if n is below a chosen threshold:
        return CONVENTIONAL-MULTIPLY(A, B)
```

```
    split A into A11, A12, A21, A22
    split B into B11, B12, B21, B22

    calculate M1, M2, M3, M4, M5, M6, M7
    calculate C11, C12, C21, C22 using reconstruction formulas
    combine four C blocks into C
    return C
```

6.6 Python Implementation

```
def conventional_multiply(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]
    return result


def add(X, Y):
    n = len(X)
    return [[X[i][j] + Y[i][j] for j in range(n)]
            for i in range(n)]


def subtract(X, Y):
    n = len(X)
    return [[X[i][j] - Y[i][j] for j in range(n)]
            for i in range(n)]


def split(M):
    n = len(M)
    mid = n // 2
    return (
        [row[:mid] for row in M[:mid]],
        [row[mid:] for row in M[:mid]],
        [row[:mid] for row in M[mid:]],
        [row[mid:] for row in M[mid:]]
    )


def join(C11, C12, C21, C22):
    top = [C11[i] + C12[i] for i in range(len(C11))]
    bottom = [C21[i] + C22[i] for i in range(len(C21))]
    return top + bottom


def strassen(A, B, threshold=2):
    n = len(A)
    if n <= threshold:
        return conventional_multiply(A, B)

    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)

    M1 = strassen(add(A11, A22), add(B11, B22), threshold)
    M2 = strassen(add(A21, A22), B11, threshold)
    M3 = strassen(A11, subtract(B12, B22), threshold)
    M4 = strassen(A22, subtract(B21, B11), threshold)
    M5 = strassen(add(A11, A12), B22, threshold)
    M6 = strassen(subtract(A21, A11), add(B11, B12), threshold)
    M7 = strassen(subtract(A12, A22), add(B21, B22), threshold)

    C11 = add(subtract(add(M1, M4), M5), M7)
    C12 = add(M3, M5)
    C21 = add(M2, M4)
    C22 = add(add(subtract(M1, M2), M3), M6)

    return join(C11, C12, C21, C22)
```

```
A = [[1, 7], [3, 5]]
B = [[1, 3], [7, 5]]
print(strassen(A, B))
```

6.7 Sample Output

```
[[50, 38], [38, 34]]
```

6.8 Recurrence and Operation Comparison

Strassen's recurrence is $T(n) = 7T(n/2) + cn^2$. By the Master Theorem, $T(n) = \Theta(n^{\log_2 7})$, where $\log_2 7$ is approximately 2.8074.

For $n = 1{,}024 = 2^{10}$, conventional multiplication performs $1{,}024^3 = 1{,}073{,}741{,}824$ scalar multiplications. The ideal recursive Strassen multiplication count is $7^{10} = 282{,}475{,}249$ scalar multiplications at the leaves. This is approximately 26.3% of the conventional count, or about 73.7% fewer scalar multiplications. This comparison excludes additions, subtractions, data copying and memory-management overhead.

6.9 Why Strassen May Not Be Faster in Practice

- Strassen performs more matrix additions and subtractions than conventional multiplication.
- Recursive splitting creates temporary matrices and additional memory traffic.
- For small matrices, recursion and allocation overhead can exceed the saved multiplication cost.
- Numerical round-off and stability must be considered when using floating-point values.
- Padding non-power-of-two dimensions can add unnecessary work.

6.10 Practical Hybrid

A practical design uses Strassen only when matrix blocks are sufficiently large and uses conventional multiplication below a tuned threshold. Production code should also pad non-power-of-two dimensions or use an implementation designed for rectangular matrices. The threshold should be selected by benchmarking on the target hardware and data type.

6.11 Module 4 Conclusion

Strassen's method reduces the asymptotic multiplication exponent from 3 to approximately 2.8074. It can reduce multiplication count for large matrices, but a hybrid strategy is more practical because it limits recursion overhead for small blocks.

7. INTEGRATED IMPLEMENTATION AND SAMPLE OUTPUTS

7.1 Processing Flow

The four modules form a pipeline. First, incoming orders are consolidated and sorted by timestamp using stable Merge Sort. The sorted list is then used by the customer-support lookup service, which applies first-occurrence Binary Search. Seller scores are refreshed using the improved Quick Sort design. Recommendation matrices are multiplied using the hybrid Strassen method.

```
Incoming Orders
      |
      v
Stable Merge Sort by timestamp
      |
      +----------------------> Audit Report
      |
      v
Sorted Order Store
      |
      v
First-Occurrence Binary Search <---- Customer Support Query

Seller Scores ---> Randomized 3-Way Quick Sort ---> Live Leaderboard

Preference Matrix A + Product Matrix B
      |
      v
Hybrid Strassen Multiplication
      |
      v
Recommendation Scores
```

7.2 Combined Demonstration Program

The following short driver demonstrates the sample inputs and expected results from the four modules. The individual function implementations are provided in their respective module sections above.

```
# Demonstration driver (run after defining the four module functions)

orders = [(105, "O1"), (102, "O2"), (105, "O3"),
          (101, "O4"), (102, "O5"), (103, "O6")]
sorted_orders = merge_sort_orders(orders)
print("Module 1:", sorted_orders)

scores = [42, 17, 63, 8, 55, 29]
print("Module 2:", quick_sort_3way(scores))

idx = first_timestamp_index(sorted_orders, 102)
print("Module 3:", idx, sorted_orders[idx])

A = [[1, 7], [3, 5]]
B = [[1, 3], [7, 5]]
print("Module 4:", strassen(A, B))
```

7.3 Expected Combined Output

```
Module 1: [(101, 'O4'), (102, 'O2'), (102, 'O5'),
           (103, 'O6'), (105, 'O1'), (105, 'O3')]
Module 2: [8, 17, 29, 42, 55, 63]
Module 3: 1 (102, 'O2')
Module 4: [[50, 38], [38, 34]]
```

7.4 Correctness Checks

Module	Test condition	Expected result
Merge Sort	Timestamps are non-decreasing	Sorted sequence; equal-key arrival order preserved

Quick Sort	Scores include unordered and repeated values	All values sorted; duplicates retained
Binary Search	Target 102 occurs more than once	Index 1, the first occurrence
Strassen	2 × 2 sample matrices	[[50, 38], [38, 34]]

The sample outputs demonstrate functional correctness for the supplied case-study data. For a production deployment, tests should also include empty input, one element, already sorted input, reverse-sorted input, duplicate-heavy values, absent search keys, and matrices of different supported dimensions.

8. COMPARATIVE ANALYSIS AND FINAL DESIGN

8.1 Algorithm Comparison

Module	Selected algorithm	Time complexity	Extra memory	Main reason
Order consolidation	Stable Merge Sort	O(n log n)	O(n)	Stable and predictable
Seller leaderboard	Randomized 3-way Quick Sort	Expected O(n log n); worst $O(n^2)$	Expected O(log n) stack	Avoids fixed first-pivot weakness; groups duplicates
Order lookup	First-occurrence Binary Search	O(log n)	O(1)	Fast lookup and earliest duplicate
Recommendation scores	Hybrid Strassen	O(n^2.807) for large blocks	Temporary block storage	Fewer recursive scalar multiplications for large matrices

8.2 Final Design Decisions

Module 1 – Retain Merge Sort, with stable tie handling. For the daily volume of 10 million orders, predictable O(n log n) performance is more important than an unstable in-place sort. If memory is insufficient, use external merge sort.

Module 2 – Replace first-element-pivot Quick Sort with randomized three-way partitioning. This avoids the predictable poor partition pattern caused by nearly sorted input. If the service requires a hard worst-case guarantee, use introsort or a balanced ranking index. If only a few scores change, incremental updates may be more efficient than full sorting.

Module 3 – Use first-occurrence Binary Search over the timestamp-sorted order store. It satisfies the requirement to return the earliest order and needs at most 24 iterations for 10 million records under the standard bound.

Module 4 – Use a hybrid Strassen implementation for large square matrix blocks, with conventional multiplication below a measured threshold. The theoretical multiplication reduction is valuable at 1,024 × 1,024, but implementation overhead must be measured.

8.3 Trade-Offs and Conditions for Change

Decision	Main trade-off	When to change
Stable Merge Sort	Uses O(n) extra memory	Use external merge sort if data exceeds RAM; consider distributed sorting for multi-node scale
Randomized 3-way Quick Sort	Expected rather than guaranteed O(n log n) time	Use introsort or balanced tree if worst-case latency is unacceptable
First-occurrence Binary Search	Requires data to remain sorted and index to be maintained	Use an indexed database lookup if data is stored persistently or updates are frequent
Hybrid Strassen	Extra additions, temporary storage and numerical considerations	Use optimized conventional/BLAS multiplication if benchmarks show it is faster

8.4 Operational Recommendations

- Maintain a timestamp-sorted order index so that lookup does not require sorting again for each request.

- Separate nightly batch processing from interactive services so that report generation does not block support lookups.
- Measure leaderboard refresh latency using realistic nearly sorted and duplicate-heavy score distributions.
- Benchmark the Strassen threshold using the actual processor, memory layout and numeric representation.
- Record correctness tests and performance measurements separately; do not confuse theoretical operation estimates with observed elapsed time.

8.5 Overall Evaluation

The redesigned architecture addresses the four stated bottlenecks with algorithms that match the data characteristics. Merge Sort protects audit ordering, Quick Sort is improved for nearly sorted leaderboard data, Binary Search accelerates support queries, and Strassen's method reduces the theoretical multiplication count for large recommendation matrices. The final solution balances asymptotic efficiency with stability, memory and implementation overhead.

9. CONCLUSION

This case study demonstrates how divide-and-conquer algorithms can improve the performance of a large retail analytics platform. The selection of an algorithm must be based not only on its average time complexity but also on the actual input characteristics and business requirements.

Stable Merge Sort is appropriate for nightly order consolidation because it guarantees O(n log n) time and preserves the original order of orders with equal timestamps. Randomized three-way Quick Sort is a safer choice than first-element-pivot Quick Sort for a nearly sorted seller leaderboard, although a worst-case-guaranteed alternative should be considered when strict latency limits apply. First-occurrence Binary Search provides O(log n) timestamp lookup while returning the earliest matching order. Strassen's method reduces the multiplication exponent to approximately 2.8074, and a hybrid implementation helps control recursion and memory overhead.

The final KartIQ design combines correctness, efficiency and practical considerations. The report also highlights that theoretical complexity is only one part of system performance. Real deployments must be validated through benchmarking, memory profiling, concurrency testing and operational monitoring.

10. REFERENCES

1. Course assessment document: CSA06 – Design and Analysis of Algorithms, CO3 Case Study – Divide-and-Conquer Engine of a Retail Data Analytics Platform (KartIQ), supplied assignment brief.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
3. Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley.
4. Strassen, V. (1969). Gaussian elimination is not optimal. Numerische Mathematik, 13, 354–356.
5. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

APPENDIX A – QUICK COMPLEXITY SUMMARY

Algorithm	Best case	Average / expected	Worst case	Space
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort (randomized)	O(n log n)	Expected O(n log n)	$O(n^2)$	Expected O(log n) stack
Binary Search	O(1) if midpoint matches	O(log n)	O(log n)	O(1)
Strassen	Θ(n^2.807)	Θ(n^2.807)	Θ(n^2.807)	Temporary block storage

APPENDIX B – SAMPLE INPUT AND OUTPUT SUMMARY

Module	Input	Output
Merge Sort	[(105,O1),(102,O2),(105,O3),(101,O4),(102,O5),(103,O6)]	[(101,O4),(102,O2),(102,O5),(103,O6),(105,O1),(105,O3)]
Quick Sort	[42,17,63,8,55,29]	[8,17,29,42,55,63]
Binary Search	Sorted timestamps; target 102	First index = 1; order O2

Strassen	A=[[1,7],[3,5]], B=[[1,3],[7,5]]	C=[[50,38],[38,34]]

APPENDIX C – SUBMISSION CHECKLIST

- Case requirements and key issues identified for all four modules.
- Algorithm choices justified with the relevant input characteristics.
- Merge Sort trace demonstrates stable ordering for duplicate timestamps.
- Quick Sort partition trace and first-pivot worst-case issue explained.
- Binary Search returns the first matching timestamp.
- Strassen's seven products and reconstruction formulas verified.
- Recurrences, complexity, operation estimates and memory trade-offs included.
- Python pseudocode, implementation and sample outputs included.